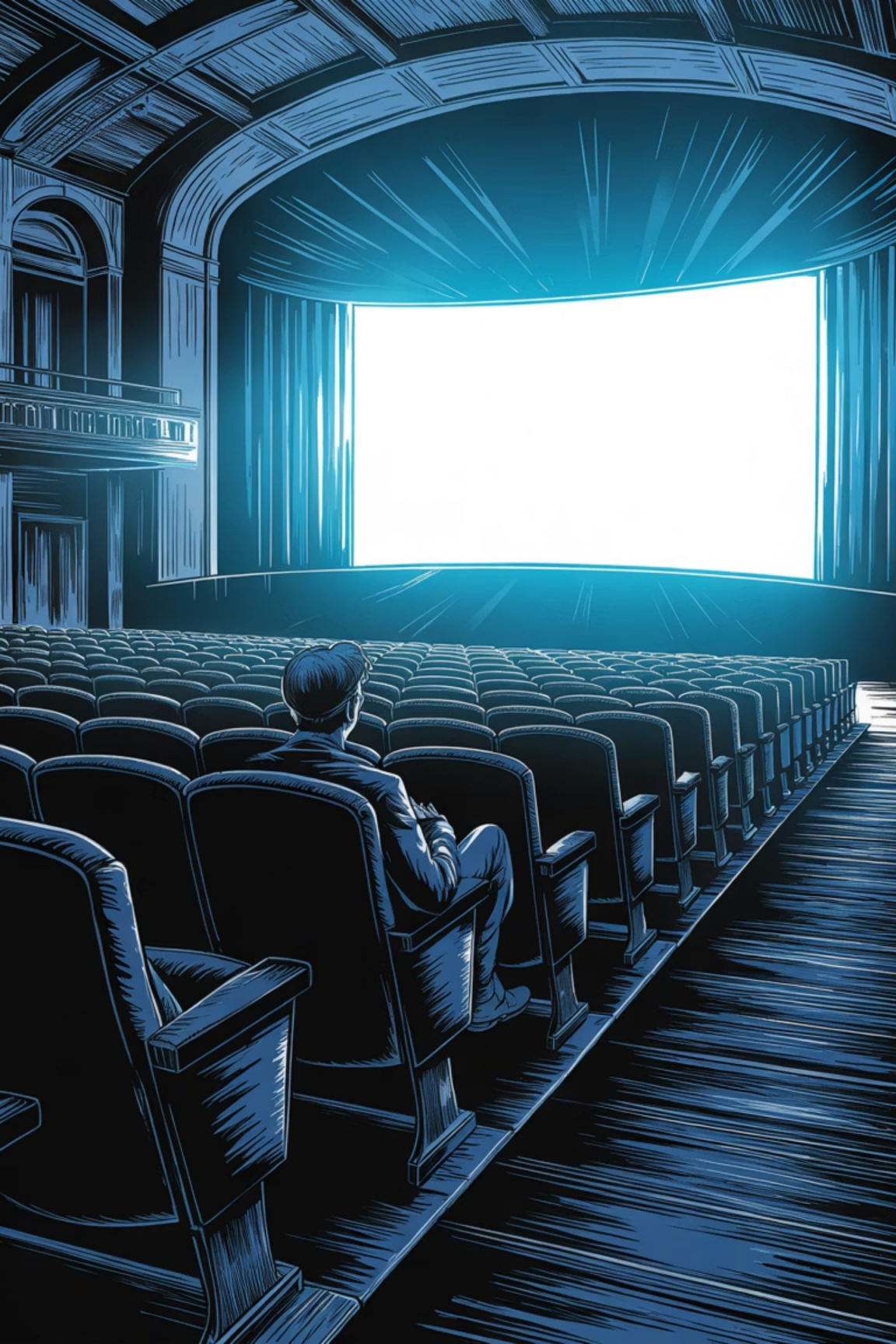




SOLID Principles

Movie Ticket Booking System in C#

PRESENTED BY

M NANDHINI

The Problem

Traditional systems pack multiple responsibilities into a **single class** — booking, payment, and notifications all mixed together. The result? Code that is hard to maintain, test, and extend.

🔧 Hard to Maintain

Changes in one area break unrelated features.

🚫 Hard to Extend

Adding a new payment method requires touching core logic.

❑ Hard to Test

Tightly coupled code resists isolated unit tests.



What Are SOLID Principles?

Five design principles that guide developers toward writing **clean, flexible, and scalable** object-oriented code.

01

Single Responsibility

One class, one job.

02

Open / Closed

Open for extension, closed for modification.

03

Liskov Substitution

Subtypes must be substitutable.

04

Interface Segregation

Small, focused interfaces.

05

Dependency Inversion

Depend on abstractions, not concretions.

One Class, One Job



Each class in the system has a **single, well-defined responsibility**. This keeps code focused and easy to reason about.

BookingService

Handles ticket booking logic only.

Payment Classes

Each handles its own payment method.

NotificationService

Sends confirmations and alerts.

O — OPEN / CLOSED

Open for Extension, Closed for Modification

The system allows **new payment methods** to be added without changing existing code. Simply implement the `IPayment` interface and plug it in.

✓ Open for Extension

Add `CreditCardPayment`, `PayPalPayment`, or any new method freely.

🔒 Closed for Modification

Existing booking and notification logic remains untouched.



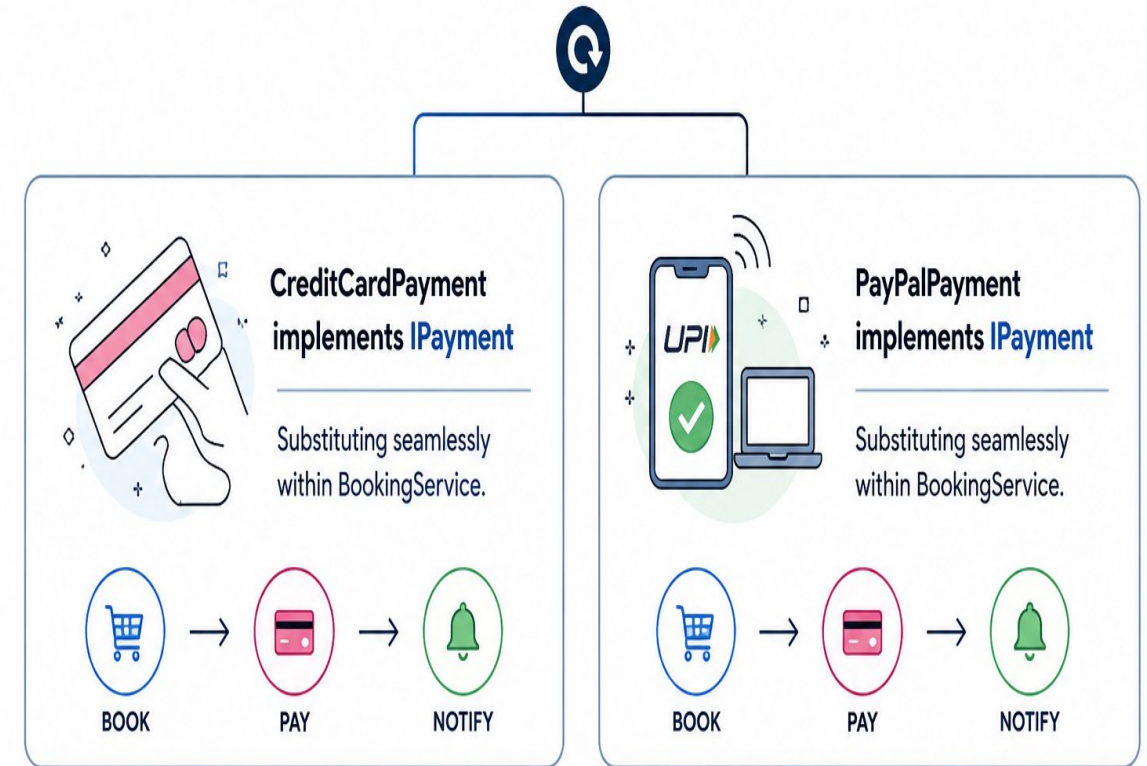
Subtypes Must Be Interchangeable

What It Means

Any implementation of `IPayment` can replace another **without affecting functionality**. The system behaves correctly regardless of which concrete class is used.

i If it looks like a payment and acts like a payment, it *is* a payment.

Subtypes Must Be Interchangeable



What it means:

Any implementation of `IPayment` can replace another without affecting functionality. The system behaves correctly regardless of which concrete class is used.



If it looks like a payment and acts like a payment, it is a payment.

Small & Focused Interfaces

Rather than one bloated interface, the system uses **small, specific interfaces**. Clients only depend on what they actually need.



IPayment

Contains only the `Pay()` method — nothing more, nothing less.



INotification

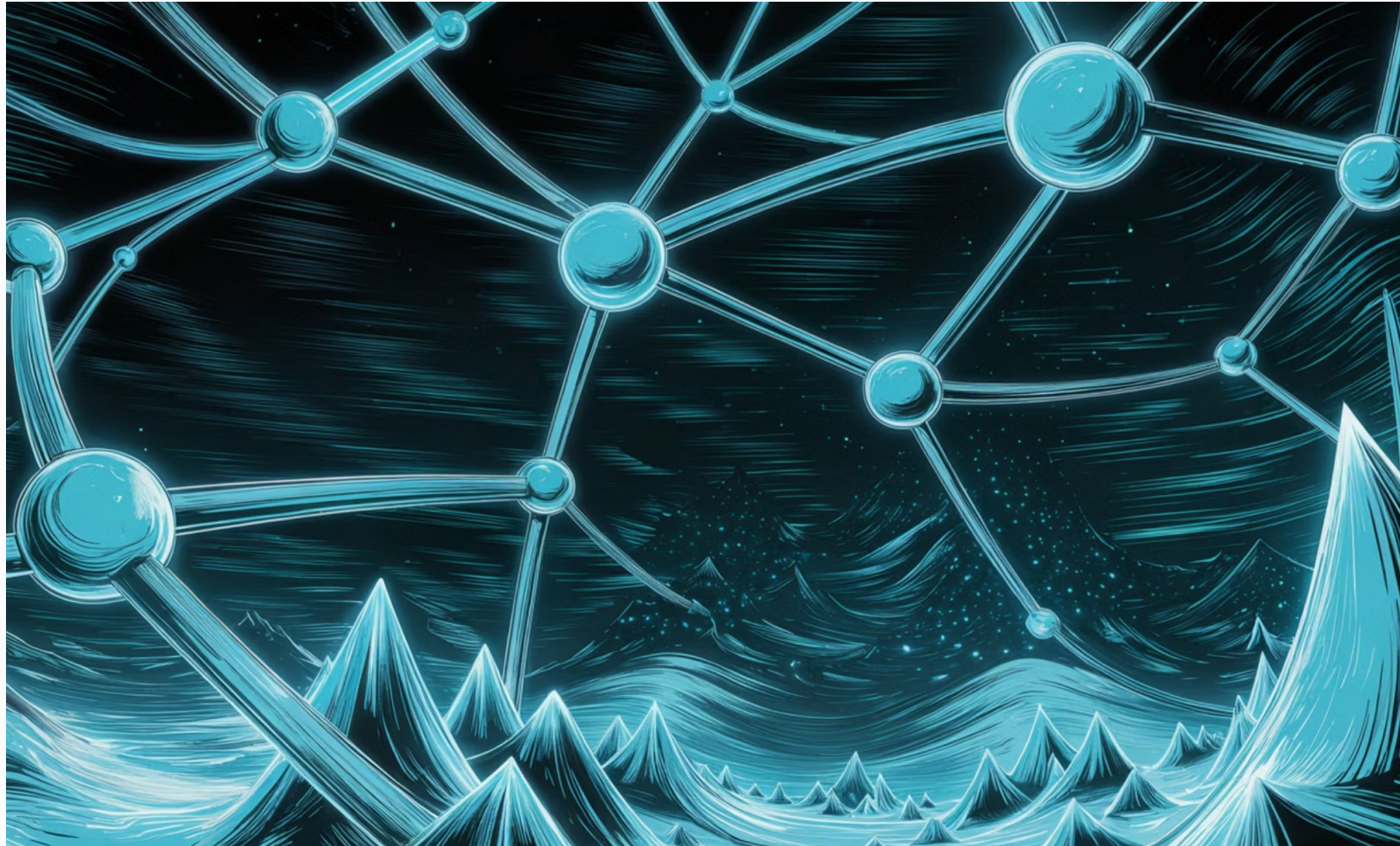
Handles only sending alerts, separate from booking or payment logic.



IBooking

Manages seat selection and reservation independently.

Depend on Abstractions



The system depends on **abstractions** (like `IPayment`) rather than concrete classes. This decouples components and makes the system flexible.

→ High-level modules

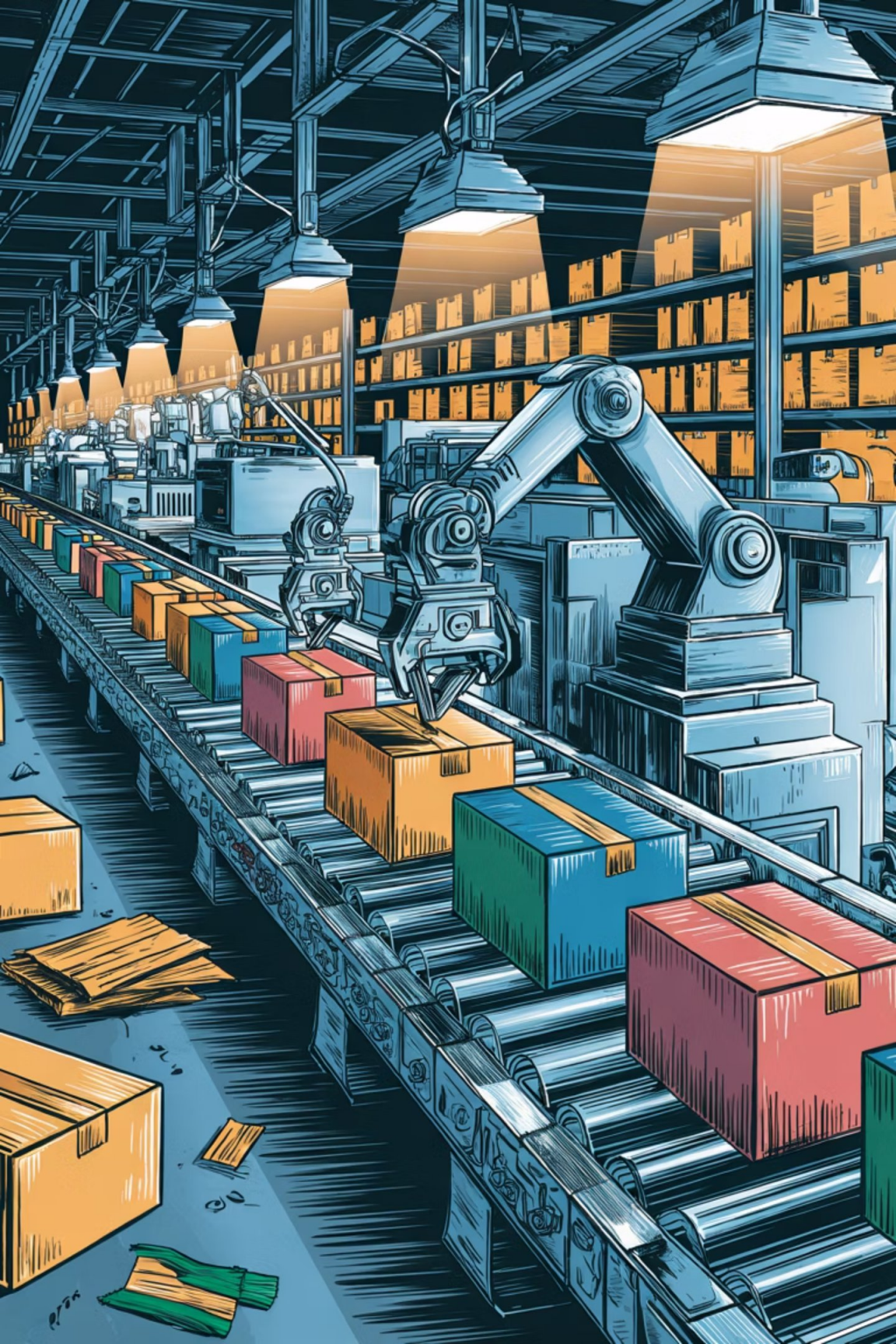
`BookingService` depends on `IPayment`, not `CreditCardPayment`.

→ Low-level modules

Concrete payment classes implement `IPayment`.

→ Result

Swap implementations without touching dependent code.



Factory Pattern at Runtime

A **Factory Pattern** dynamically selects the correct payment method at runtime based on user input — no hard-coded conditionals, no messy switch statements.

User Selects

Credit Card, PayPal, or UPI at checkout.

Factory Resolves

Returns the correct `IPayment` implementation.

System Executes

Processes payment with zero coupling to concrete types.

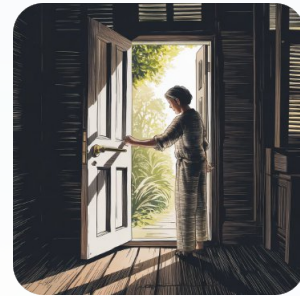
Conclusion

Applying SOLID principles transforms a fragile, tangled system into one that is **flexible, scalable, and maintainable**.



Single Responsibility

Each class owns one concern.



Open / Closed

Extend without rewriting.



Liskov Substitution

Subtypes swap freely.



Interface Segregation

Lean, focused contracts.



Dependency Inversion

Abstractions over concretions.

✅ The result: a Movie Ticket Booking System built to grow, not break. New features can be added without modifying existing code.

THANK YOU!